

PLANAZO

La excusa perfecta para salir.

React 19 Node.js 22 PostgreSQL 16 Docker

MEMORIA TÉCNICA

Desafío de tripulaciones

Desarrollo web FullStack · 2026

Alumnos:

Saray Guillen
Jon Aldekoa
Montse García
Ermidio Figueiredo

1. ¿Qué es Planazo?

Planazo es una aplicación web social que permite a usuarios descubrir actividades y negocios de ocio, crear grupos de amigos, proponer planes con actividades concretas y votar colectivamente cuál realizar. El objetivo es eliminar la fricción de la organización grupal: de la idea al plan confirmado en pocos pasos.

2. Arquitectura general

La aplicación se despliega con tres contenedores Docker orquestados mediante docker-compose. El navegador solo interactúa con Nginx; el backend nunca expone puertos al host directamente.

```
[Navegador]
|
| HTTP :80
v
[ Nginx (Docker) ]  <- SPA de React (build estático)
| /api/* -> proxy_pass -> [ Backend Node.js :3000 ]
|
| [ PostgreSQL 16 ]
```

Contenedor	Imagen base	Función
postgres	postgres:16	BD relacional. Datos persistidos en volumen postgres_data.
backend	node:22-alpine	API REST Express 5. No expone puertos al host.
nginx	nginx:alpine	Sirve el SPA y hace proxy de /api/* al backend. Puerto 80.

Redes Docker separadas por capas

- **backend-network:** postgres ↔ backend (BD nunca accesible desde fuera)
- frontend-network: backend ↔ nginx
- shared_uploads: volumen compartido para imágenes subidas por usuarios

3. Repositorio y flujo Git / GitHub

El repositorio raíz actúa como repositorio paraguas que gestiona dos git submodules independientes, cada uno con su propio historial y repositorio en GitHub. Todos los repos son privados por seguridad.

Submodule	Repositorio GitHub	Ruta local
frontend	Jaldekoa/frontend	./frontend
backend	Jaldekoa/backend	./backend

Estrategia de ramas

La rama base de trabajo es dev en ambos submodules. La rama main se reserva para las entregas finales.

Rama	Propósito	Ejemplo real
main	Entrega / producción	—
dev	Integración continua del equipo	—
DES-XX/descripcion	Feature branch (1 tarea de Jira)	DES-26/middleware-auth

Flujo de trabajo

- Cada miembro crea una rama DES-XX/<tarea> desde dev.
- Abre un Pull Request hacia dev.
- El PR se revisa y se hace merge.
- Periódicamente se mergea dev → main para la entrega.

4. Docker y cómo está resuelto

Dockerfile del Frontend (multi-stage build)

```
# Stage 1 — Build
FROM node:22-alpine AS builder
WORKDIR /app
COPY package*.json .
RUN npm install
COPY . .
ARG VITE_API_URL=/api
RUN npm run build          # -> /app/dist

# Stage 2 — Serve
FROM nginx:alpine
COPY --from=builder /app/dist /usr/share/nginx/html
COPY nginx.conf /etc/nginx/conf.d/default.conf
```

La variable VITE_API_URL=/api se inyecta en build-time para que todas las llamadas al backend usen ruta relativa, sin necesidad de configurar CORS.

nginx.conf — configuración de proxy

```
location /api/ {
    proxy_pass          http://backend:3000/; # strip /api prefix
    proxy_set_header    X-Real-IP $remote_addr;
    proxy_set_header    X-Forwarded-For $proxy_add_x_forwarded_for;
}
```

El frontend llama a /api/activity; Nginx lo transforma en http://backend:3000/activity eliminando el prefijo /api.

Inicialización de la base de datos

El directorio ./backend/db/ se monta en el punto de entrada de Docker (/docker-entrypoint-initdb.d). El fichero 01_schema.sql se ejecuta automáticamente la primera vez que se crea el contenedor de Postgres.

5. Backend

Node.js 22 con ESM nativo ("type": "module") y Express 5. La arquitectura sigue un patrón:

Controller → Service → Model (Sequelize).

Estructura de carpetas (src/)

Carpeta	Responsabilidad
config/	db.js (Sequelize) + options.js (CORS, rate-limit, JWT_SECRET)
controllers/	Lógica HTTP separada por recurso
middlewares/	auth (JWT cookie) · role (RBAC) · validator (express-validator)
models/	Modelos Sequelize + index.js con todas las relaciones
routes/	router.js central + sub-routers por recurso
schemas/	Validaciones Zod (chatbot Nora)
services/	Lógica de negocio y acceso a datos
utils/	Logger Morgan + Multer (upload de ficheros)

Dependencias clave y justificación

Librería	Justificación
sequelize + pg	ORM con PostgreSQL. Permite modelar relaciones M:M con tablas pivote.
jsonwebtoken + bcrypt	Autenticación JWT en httpOnly cookie. Protege contra XSS.
express-rate-limit	100 req / 5 min por IP. Protección básica contra abuso de la API.
helmet	Cabeceras de seguridad HTTP (CSP, HSTS...) en una línea.
multer	Gestión de subida de ficheros (imágenes de negocios, avatares).
zod + express-validator	Doble capa de validación: esquemas Zod + middleware por ruta.

Autenticación

El JWT se almacena en una httpOnly cookie (nunca en localStorage) para evitar acceso desde JavaScript malicioso. El middleware auth.middleware.js extrae req.cookies.token y verifica con jwt.verify. El middleware de rol requireRole(...roles) comprueba req.user.role para implementar RBAC.

6. Base de datos

PostgreSQL 16. El esquema se define en backend/db/01_schema.sql y se aplica automáticamente al arrancar el contenedor por primera vez. Se usa integridad referencial (Foreign Keys) para evitar registros huérfanos.

Tabla	Descripción
users	Usuarios: nombre, email, password hash, birth_date, role ENUM, avatar_url
businesses	Negocios: nombre, dirección, coordenadas GPS, redes sociales, logo, banner
activities	Actividades de un negocio: precio, fecha, slots, indoor/outdoor, QR, imagen
activity_categories	Categorías de actividades (tabla maestra)
groups	Grupos de amigos
members	Tabla pivote users ↔ groups
plans	Plan creado en un grupo, asociado a usuario y fecha
proposals	Tabla pivote plans ↔ activities (actividades propuestas)
votes	Votos de un miembro sobre una propuesta (tipo ENUM vote_option)
reviews	Reseñas de un usuario sobre un negocio, con categoría y rating
review_categories	Categorías de reseña (tabla maestra)
user_preferences	Tabla pivote users ↔ activity_categories (preferencias del usuario)

Relaciones ORM destacadas

- User ↔ Group: M:M vía Member
- User ↔ ActivityCategory: M:M vía UserPreferences
- Business → Activity: 1:N
- Plan ↔ Activity: M:M vía Proposals
- Member → Vote → (Plan + Activity): el voto se hace sobre una propuesta concreta

7. Frontend

React 19 con Vite 8. La API se consume a través de una capa de servicios propia (src/services/) que centraliza todos los fetch a /api/*. El compilador de React (babel-plugin-react-compiler) optimiza re-renders automáticamente.

Páginas principales

Página / Grupo	Función
HomePage	Home con actividades recomendadas por el motor de Data Science
SearchList / SearchMap	Búsqueda de actividades por lista o mapa interactivo (Leaflet)
ActivityDetailPage	Detalle de actividad con galería, precio, slots y reserva
BusinessDetailPage	Perfil público de un negocio con sus actividades
GroupsPage / CreateGroupPage	Gestión de grupos de amigos
GroupPlanPage / CreatePlanPage	Flujo de propuesta y votación de planes
GroupChatPage	Chat grupal
ProfilePage / ProfileEditPage	Perfil de usuario y edición
NoraChatPage	Interfaz del chatbot Nora (microservicio externo)
GuardadosPage / RewardsPage	Funcionalidades secundarias: guardados y recompensas

Dependencias clave

Librería	Uso
react-leaflet + leaflet	Mapa interactivo para búsqueda geolocalizada de actividades
react-router-dom v7	Enrutamiento con layout raíz MainLayout y rutas autónomas
lucide-react	Iconografía consistente con tree-shaking automático
react-datepicker	Selector de fechas para creación de planes
React Compiler	Optimización automática de re-renders sin memo manual

8. Servicios externos y variables de entorno

Motor de recomendaciones (Data Science)

Microservicio externo alojado en MODEL_HOST. El backend actúa como proxy transparente hacia él.

- GET / — health check
- POST /actualizar-pesos
- POST /recomendaciones

Variables de entorno

En desarrollo local se usa un fichero .env en backend/. En Docker las inyecta directamente docker-compose.yml en la sección environment.

Variable	Descripción
POSTGRES_*	Credenciales y nombre de la base de datos
APP_PORT	Puerto del servidor Express (por defecto 3000)
JWT_SECRET	Clave de firma de tokens JWT
CHATBOT_HOST/PORT	Dirección del microservicio chatbot Nora
MODEL_HOST	Dirección del microservicio de recomendaciones
FRONT_PORT	Puerto del frontend para configurar CORS en desarrollo

9. Conclusiones

Planazo surge del desafío que nos propuso la empresa Inetum, dándonos un público objetivo concreto, personas de 20 años a 45 años, sin hijos, naturales de Euskadi y cuyas actividades sociales son de tipo urbanita, gente currante que quiere desconectar y hacer actividades encaminadas al ocio.

La idea de esta aplicación fue hacer primeramente un Producto Mínimo Viable (MVP), por los plazos establecidos y poder entregar un producto sencillo y funcional. Hemos invertido muchas horas que no siempre fueron fáciles, pero aprendido mucho a la vez, lo que parece a simple vista muy vistoso, no siempre es sencillo, y es importante hacer las cosas bien, se debe usar la IA pero de manera responsable, aunque parezca que puede hacer magia en el momento. Somos conscientes que estamos en mundo en constante cambio con la tecnología, y que la manera de poder avanzar es adaptarnos al cambio, por eso hemos venido a programar, y afrontar los desafíos que nos depare el futuro, si surgen problemas pues hay que resolver esos problemas, aunque estos sean increíblemente tediosos, esa es la vida del programador.

El verdadero reto no fue técnico: fue humano. Coordinar un equipo, mantener el ritmo cuando las cosas no salen, y aprender a sacar el trabajo adelante a pesar de las diferencias. Eso es lo que realmente se lleva de un proyecto así, el aprendizaje y la experiencia adquirida. Es por eso que con constancia, y perseverancia se pueden lograr grandes cosas.

También agradecer especialmente a los profesores de nuestra vertical de Desarrollo web FullStack:

Danel Lafuente y Jesús Cabado por su enorme paciencia y disposición. Personas que valen muchísimo y que se preocupan de verdad por sus alumnos.